`tests/ddos/proxy_flood_test.sh` — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Authored + parse-clean. Runs live against the running proxy (HTTP forward `localhost:34128`, SOCKS5 `localhost:34080`); honest `SKIP` (§11.4.3) when the proxy is absent or the target is unreachable both ways.

> Companion (§11.4.18) to the in-source documentation block at the top of the script.

Overview

§11.4.169 **DDoS / load-flood** test for the LIVE proxy. It sends a **bounded high-rate burst** (default `300` requests across `20` parallel workers) through the HTTP forward proxy and proves the proxy **degrades gracefully** rather than **collapses**: under overload it MAY shed load (`503`/`429`/timeout/reset) but it MUST stay UP — the process MUST NOT crash, both listeners MUST stay bound, and normal `204`/`200` service MUST **recover** the moment the flood ends (§11.4.85 resource-exhaustion: refuse cleanly OR degrade, **never** crash).

Every flood outcome is categorised into a captured **census** — `success` | `refuse` (clean load-shed) | `timeout` | `error` (connection reset). Load-shedding during the flood is **acceptable and healthy**; the decisive proof is the **post-flood recovery request succeeding** plus **both listeners surviving** (§11.4.108 runtime-signature, §11.4.69 captured evidence). A metadata-only / "no error" PASS is refused (§11.4.1); a crash / hung recovery / dropped listener is a `FAIL`; an absent proxy is an honest `SKIP` — never a fabricated PASS.

Distinct from its siblings (no duplication): `tests/stress/proxy_forward_stress.sh` is the **stress** case where *every* request must succeed; this is the **overload / degrade-not-collapse** case where shedding is allowed and *survival* is mandatory. `tests/dynamic/suites/ddos_flood_suite.sh` floods the `dynamic` VPN-aware **routing** stack (`target-a.internal`); **this** test floods the **real proxy**

ports. `tests/regression/ddos_flood_evidence_test.sh` is a **pure-function anti-bluff harness guard** (no network); **this** is a **live** flood.

Prerequisites

- Committed library `tests/lib/evidence.sh` (sourced — provides `_code_in`, `port_is_listening`, `ab_pass_with_evidence`, `ab_skip_with_reason`, `_evidence_emit`).
- `curl`, `awk`, `sort`, `grep`, `uniq`, POSIX `sh`, `date`.
- A running proxy on `HTTP_PROXY_PORT` (and, for the SOCKS survival probe, `SOCKS_PROXY_PORT`) for a non-`SKIP` run.
- Write access to `qa-results/` (gitignored).

Usage examples

- Default flood against `localhost:34128` / `localhost:34080`: `bash tests/ddos/proxy_flood_test.sh`
- Conductor invocation under host-safety caps (§12.6 — the hard constraint): `GOMAXPROCS=2 nice -n 19 ionice -c 3 bash tests/ddos/proxy_flood_test.sh`
- Heavier bounded burst (still hard-capped `total<=500`, `conc<=30`): `FLOOD_TOTAL=400 FLOOD_CONC=30 bash tests/ddos/proxy_flood_test.sh`

Env knobs: `HTTP_PROXY_URL` (default `http://localhost:34128`), `HTTP_PROXY_PORT` (default `34128`), `SOCKS_PROXY_HOST` (default `localhost`), `SOCKS_PROXY_PORT` (default `34080`), `FLOOD_TARGET` (default `https://www.gstatic.com/generate_204`), `FLOOD_EXPECT` (default `204 200`), `FLOOD_TOTAL` (default `300`, hard-capped `500`), `FLOOD_CONC` (default `20`, hard-capped `30`), `FLOOD_MAX_TIME` (default `5` s, capped `30`), `FLOOD_EVIDENCE_DIR` (default `qa-results/ddos/proxy_flood_<ts>`).

Host safety (§12.6 — non-negotiable)

The burst is **hard-bounded regardless of env**: total requests `<= 500`, parallel workers `<= 30`, per-request `--max-time` (default `5` s) so no request hangs unboundedly. Malformed knob values fall back to the safe defaults (§11.4.6 — never trust an unparseable knob to widen the blast radius). Worst-case wall-clock is `PER_WORKER × MAX_TIME` (degenerate all-timeout case) — finite and bounded. The flood pressures the **proxy**, not host memory; the tool is shell + `curl` only, well under the §12.6 60 % host-memory ceiling. Memory-leak-over-soak is **out of scope** here (honest boundary, §11.4.6) — that is `tests/dynamic/suites/memory_soak_suite.sh`.

Edge cases

- **Real flood landed + both listeners survived + recovery `204/200`** → `PASS`, citing `flood.evidence`. Exit `0`. (Load-shed `503`/`429`/timeout during the flood is fine.)
- **SOCKS listener was up before the flood and DOWN after** → `FAIL` (a service collapsed under load). Exit `1`.
- **Recovery fails (or HTTP listener dropped) BUT the target is reachable directly** → `FAIL` — a real proxy crash / collapse / stuck-fail-closed (§11.4.68 no fail-open). Exit `1`.
- **Proxy up + recovers but ZERO flood requests landed** (all timeout/error) → `SKIP` (`network_unreachable_external`) — no real flood pressure was applied, so "survived" would be vacuous (§11.4.69 / §11.4.1). Exit `3`.
- **Recovery + direct probe both fail, port still listening** → `SKIP` (`network_unreachable_external`) — a third-party outage is not a proxy defect (§11.4.1). Exit `3`.
- **HTTP proxy port not listening pre-flood** → `SKIP` (`topology_unsupported`) before flooding nothing. Exit `3`.

Internal behaviour

- `#!/usr/bin/env bash`, POSIX-clean body (`sh -n` + `bash -n`, §11.4.67).
- Pre-flood listener snapshot (`port_is_listening` on both ports) → `flood.evidence`; proxy-absent short-circuits to an honest `SKIP`.
- Flood: `FLOOD_CONC` bounded parallel workers, each issuing `PER_WORKER` requests, appending `%{http_code} %{time_total}` per request to its **own** file (a verdict never rests on a background job's exit status).
- `classify_outcome` (pure, no network) buckets each request into `success`/`refuse`/`timeout`/`error`; census + distinct-code histogram → `census.txt`; `success + refuse = flood_landed` (measurable HTTP responses = positive flood evidence).
- Post-flood survival: both listeners re-probed, a recovery request through the HTTP proxy, a direct cross-check probe, and a single SOCKS5 recovery probe → `flood.evidence`.
- `trap ... EXIT INT TERM` reaps **only** this script's worker PIDs (never `kill 0`) and removes the scratch dir (§11.4.14).
- Resources: shell + curl only, concurrency hard-capped, well under the §12.6 60 % host-memory ceiling.

Related

- `tests/lib/evidence.sh` — sourced anti-bluff evidence library.
- `tests/stress/proxy_forward_stress.sh` — sustained/concurrent STRESS suite (every request must succeed) that this DDoS/overload test complements.

- `tests/chaos/proxy_restart_recovery.sh` — sibling chaos recovery suite.
- `tests/dynamic/suites/ddos_flood_suite.sh` — routing-stack flood suite; `tests/regression/ddos_flood_evidence_test.sh` — the anti-bluff flood-evidence harness guard.
- Constitution §11.4.169 / §11.4.85 / §11.4.108 / §11.4.69 / §11.4.1 / §11.4.68 / §11.4.3 / §12.6.

Last verified

2026-07-01 — authored; `sh -n` + `bash -n` parse-clean; pure `classify_outcome` categorisation unit-driven (11/11 buckets correct, no network). Live flood + survival/recovery is exercised by the conductor against the running proxy.